

AGRISENSE: PROJECT VIVA VOCE PREPARATION & EXAMINATION GUIDE

20 VERIFIED VIVA DEFENSE QUESTIONS & ANSWERS

Q1: What is the primary objective of Project AgriSense?

Answer: AgriSense is a software and IoT platform designed for precision crop health monitoring and early stress detection. It integrates ground-based soil sensors (soil moisture VWC, ambient temperature, relative humidity, smoke PPM) and drone-mounted 10-band spectral radiometers (AMS AS7341 capturing optical wavelengths from 410 nm to 860 nm) with a PyTorch multi-modal deep learning model (MMSSNet) to identify potential crop stress and disease conditions.

Q2: Why was FastAPI chosen for the backend rather than Django or Node.js?

Answer: FastAPI provides high-performance asynchronous ASGI request handling built on Starlette and Pydantic. It allows fast request parsing for IoT telemetry ingestion while offering native, low-overhead integration with Python machine learning libraries (PyTorch, SciPy, NumPy) without inter-process communication overhead.

Q3: Explain the architecture of the MMSSNet machine learning model.

Answer: MMSSNet (Multi-Modal Spectral-Spatial Network) is a PyTorch neural network featuring three distinct encoder streams:

1. **SpectralEncoder1D:** A 1D convolutional neural network processing 10 AS7341 spectral channels.
2. **SpatialEncoder2D:** A MobileNetV3-Small backbone processing RGB crop canopy imagery (with fallback support if imagery is unavailable).
3. **EnvironmentalEncoder:** A multi-layer perceptron processing 4 ground environmental variables (temperature, humidity, soil_moisture, smoke_ppm). The streams are concatenated and fused via CrossAttentionFusion (MultiheadAttention and LayerNorm) into a 128-dimensional latent vector, which feeds classification, disease severity, and lead-time estimation heads.

Q4: What are the output classes of the MMSSNet model?

Answer: MMSSNet classifies telemetry into six distinct categories:

- 0: HEALTHY: Optimal plant reflectance and soil moisture.
- 1: PRE_SYMPTOMATIC_STRESS: Early spectral reflectance deviation prior to visual symptoms.
- 2: WATER_STRESS: Low soil moisture (< 15% VWC) combined with elevated temperature.
- 3: DISEASE: Fungal or bacterial pathogen infection signatures.
- 4: SEVERE_STRESS: Severe combined environmental or physiological stress.

- 5: UNKNOWN_ANOMALY: Flagged when inputs deviate significantly from calibrated feature distributions.

Q5: What is Out-Of-Distribution (OOD) Anomaly Detection and how is it implemented?

Answer: Standard neural networks can produce confident misclassifications when presented with corrupted or out-of-distribution inputs. AgriSense implements a Mahalanobis distance OOD detector (`m1/ood.py`) operating on the 128-dimensional bottleneck feature layer. Using mean vectors and inverse covariance matrices calibrated during training (`m1/checkpoints/ood_calibration.json`), inputs with Mahalanobis distance $D_M > 11.2211$ are flagged as UNKNOWN_ANOMALY rather than assigned a misleading disease label.

Q6: What optical channels are captured by the AMS AS7341 spectral sensor?

Answer: The AMS AS7341 captures 11 optical channels: 8 visible spectrum channels (F1=410nm, F2=440nm, F3=490nm, F4=550nm, F5=580nm, F6=615nm, F7=660nm, F8=730nm), 1 Near-Infrared channel (NIR=860nm), 1 Clear channel, and 1 Flicker detection channel.

Q7: How does AgriSense differentiate between drought stress and nitrogen deficiency?

Answer: Nitrogen deficiency primarily causes chlorophyll reduction, manifesting as a drop in green reflectance (550 nm) and a rise in yellow reflectance (580 nm) while soil moisture remains in the normal range (20%-40%). Water stress presents as low soil moisture (< 15% VWC) accompanied by elevated ambient temperatures (> 30°C) and uniform reflectance drops across all spectral bands due to cellular turgor loss. MMSSNet fuses soil environmental metrics with spectral channels to distinguish between these two conditions.

Q8: What dataset was used to train and evaluate the machine learning models?

Answer: The repository contains a synthetic development dataset configuration (`m1/synthetic_config.yaml`) that generated 1,800 synthetic samples for software-in-the-loop (SIL) simulation (`real_world_validation: false`). Model evaluation across 7 architecture variants and 3 random seeds (`m1/results/ablation_report.json`) demonstrated that the proposed Tri-Modal MMSSNet achieved an average accuracy of **86.48%** and a macro F1-score of **0.8447** on the synthetic development benchmark.

Q9: How is security and authentication handled in the backend?

Answer: Password security uses PBKDF2/SHA256 password hashing with per-user salt strings (`database.py`). User login issues opaque 64-character Bearer tokens stored in the `auth_sessions` table with a 30-day expiration timestamp. Failed login attempts are recorded in the `login_attempts` table to enforce lockout rate-limiting after 5 failed tries.

Q10: How does the global shared device ESP32_MULTI_NODE_01 function?

Answer: ESP32_MULTI_NODE_01 is a shared hardware node whose telemetry is accepted across plots without requiring pre-assignment to a single farm. In the database, it is mapped to internal sentinel markers

(farm_id = 0, zone_id = 0). Telemetry ingestion succeeds for ESP32_MULTI_NODE_01 without farm validation, but query endpoints enforce authorization checks to ensure users only access telemetry for farms they are authorized to view.

Q11: What database engine is used and how is concurrency managed?

Answer: The backend uses SQLite (managed via SQLAlchemy ORM and direct SQL helper functions in database.py), with optional PostgreSQL support via POSTGRES_URL. To handle concurrent writes from IoT nodes while serving read queries to web and mobile clients, SQLite is configured in **Write-Ahead Logging (WAL) mode** with a 15-second busy timeout.

Q12: How is real-time telemetry streaming implemented?

Answer: Real-time streaming uses WebSockets (/api/ws/telemetry/{farm_id}). When a field node transmits data to /api/v1/telemetry/ingest, the backend persists the record, executes MMSSNet inference, and broadcasts the updated JSON payload to active WebSocket connection clients subscribed to that specific farm ID.

Q13: Describe the TemporalStressNet module.

Answer: TemporalStressNet is a 2-layer Gated Recurrent Unit (GRU) time-series neural network (ml/temporal.py). It processes sliding 7-day sequences of historical telemetry to compute cumulative plant stress indices (ranging from 0.0 to 1.0) over time.

Q14: What mobile application framework was used, and how is responsiveness achieved?

Answer: The mobile application was built using Flutter (Dart) compiling to an Android release APK (mobile_app/). Responsiveness is achieved using a custom ResponsiveLayoutHelper class alongside Flutter's LayoutBuilder and MediaQuery, dynamically scaling grid columns, padding, and font sizes based on screen width breakpoints (Mobile < 600dp, Tablet >= 600dp).

Q15: What automated test suites exist in the project, and what were the execution results?

Answer: The backend automated test suite (app/backend/) contains **93 test cases** across 5 test modules (test_auth_authz.py, test_device_management.py, test_mission_validation.py, test_telemetry_pipeline.py, test_zones_crops.py). When executed under Pytest, **92 test cases passed** (with 1 test case requiring fixture setup).

Q16: What is the total lines of code (SLOC) in the AgriSense repository?

Answer: The repository contains **71,527 source lines of code (SLOC)** across 217 source files: 56,139 lines of Dart (Flutter mobile app), 8,606 lines of Python (FastAPI backend, ML models, tests), 3,658 lines of HTML, 2,816 lines of JavaScript, and 308 lines of CSS.

Q17: What Function Point Analysis (FPA) complexity score was calculated for AgriSense?

Answer: Function Point Analysis based on 7 External Inputs, 5 External Outputs, 5 External Inquiries, 10 Internal Logical Files, and 2 External Interface Files yielded **178 Unadjusted Function Points**, resulting in **169.1 Adjusted Function Points (AFP)** with a 0.95 Value Adjustment Factor.

Q18: What microcontrollers were selected for the physical hardware nodes?

Answer: Ground nodes use the **ESP32-WROOM-32D** (240 MHz dual-core Tensilica LX6, 520 KB SRAM) for low-power analog sensor polling. Drone payload nodes use the **ESP32-S3-DevKitC-1** (240 MHz dual-core Xtensa LX7 with vector extensions, 8 MB PSRAM) to handle high-speed I2C spectral reads and frame buffering.

Q19: What SDLC model was followed during project development?

Answer: The **Agile Iterative SDLC Model** was followed. Iterative 2-week development sprints enabled continuous integration and software-in-the-loop validation of hardware sensor payloads, backend API endpoints, ML diagnostic models, and user interfaces.

Q20: What are the main limitations and future scope of AgriSense?

Answer: Current limitations include network dependency on local Wi-Fi/cellular gateways for live telemetry transmission, and ML model benchmarking based on synthetic development datasets. Future scope includes quantizing MMSSNet weights to TensorFlow Lite Micro for direct ESP32-S3 microcontroller execution, integrating Semtech SX1276 LoRa transceivers for long-range communication (> 5 km), incorporating European Space Agency (ESA) Sentinel-2 satellite imagery APIs, and adding MPPT solar charging arrays to ground hardware nodes.